

AgentSecBench: A Benchmark Framework for Evaluating Security Tools in the Age of Agentic Development

Draft v0.1 — February 2026

Abstract

The emergence of AI coding agents has collapsed the latency window that traditional application security depended on. Static analysis, dependency scanning, and audit-layer tools were architected for human-speed development—they assume code exists before inspection. Agents generate, execute, and deploy before scanners run. Meanwhile, a new generation of AI-powered security tools—exemplified by Anthropic’s Claude Code Security—promises reasoning-based vulnerability detection that surpasses pattern-matching approaches. Yet no standardized benchmark exists to evaluate these tools against each other or against the new attack surfaces that agentic development introduces.

We propose **AgentSecBench**, an open benchmark framework that evaluates security tools across four trust dimensions: *Secure* (vulnerability detection), *Competent* (finding quality and actionability), *Accountable* (explainability and auditability), and *Surface* (coverage of novel agentic attack surfaces including MCP protocols, skill registries, and client-side exposure). We describe the benchmark architecture, a curated dataset of 60 intentionally vulnerable modern web applications, and a scoring methodology designed to capture the fundamental differences between white-box source analysis, black-box runtime reconnaissance, and traditional static/dynamic tools.

1 Introduction

Application security tooling is undergoing a phase transition. For two decades, the security industry operated on a stable assumption: developers write code, commit it, and scanners inspect it before or shortly after deployment. SAST tools match code against known vulnerability patterns. SCA tools check dependency manifests against CVE databases. DAST tools probe running applications against known attack signatures. The entire audit layer depends on a temporal gap between code creation and code execution.

AI coding agents have eliminated that gap.

Tools like Claude Code, Cursor, and Windsurf generate, test, and deploy code in seconds. The hallucinated npm package is installed before the scanner’s next scheduled run. The misconfigured API route is live before the PR reviewer wakes up. Prevention—the foundational promise of DevSecOps—requires a window of time that no longer exists.

Simultaneously, AI is being applied to security itself. Anthropic’s Claude Code Security, launched in February 2026, uses Claude Opus 4.6 to reason about codebases “the way a human security researcher would,” claiming over 500 vulnerabilities found in production open-source projects that had evaded decades of expert review [1]. This represents a fundamentally different approach: not pattern matching, but semantic understanding of code be-

havior.

Yet the security landscape is not monolithic. White-box source analysis, no matter how sophisticated, cannot detect what is already exposed to the internet through deployed client-side bundles. A hardcoded API key in a minified JavaScript file, an undocumented admin endpoint visible in a React SPA’s routing table, a source map leaking original TypeScript to any browser that requests it—these are runtime exposure problems, not source code problems. Tools operating in the black-box reconnaissance space—exemplified by automated SPA analysis agents that render applications, fetch JavaScript bundles, and extract secrets and endpoints—address a complementary attack surface that source-level tools are structurally blind to.

The problem is that no benchmark exists to evaluate any of this. We have SWE-bench for coding capability [2]. We have nothing equivalent for security tooling in the agentic era. Vendors make claims. Buyers have no basis for comparison.

This paper proposes one.

2 Background and Motivation

2.1 The Collapse of the Audit Layer

Traditional application security follows a linear pipeline:

Write → Commit → Scan → Review →

Deploy

Every commercial security tool inserts itself somewhere in this pipeline. SAST operates between Commit and Review. DAST operates after Deploy. SCA operates at Commit time by inspecting manifests. The entire model assumes sequential, human-paced development.

Agentic development compresses this to:

Prompt → Generate → Execute

There is no commit phase. There is no review window. The agent generates code and runs it. By the time a scanner processes the output, the code is already executing—potentially in production.

This is not a speed problem solvable with faster scanners. It is an architectural mismatch. The audit layer assumes code exists as a static artifact available for inspection. Agents produce code as a transient byproduct of execution. The inspection model itself is obsolete.

2.2 The Rise of AI-Powered Security

Claude Code Security represents the first major attempt by a foundation model provider to apply frontier reasoning capabilities to vulnerability detection. Rather than matching patterns against a database of known exploits, it reads code and reasons about data flows, component interactions, and logical invariants—the way an experienced security researcher would.

This approach addresses a long-standing limitation of SAST: traditional tools excel at finding known vulnerability patterns (SQL injection via string concatenation, XSS via unsanitized output) but fail at business logic flaws, broken access control, and novel vulnerability classes that don't match existing signatures [1].

However, this capability operates strictly in the white-box domain. It requires access to source code. It cannot assess what an attacker sees from the outside.

2.3 The Black-Box Reconnaissance Gap

Modern single-page applications ship substantial application logic to the client. A React or Next.js application's JavaScript bundles contain:

- API endpoint URLs and routing patterns
- Authentication flow logic and token handling
- Hardcoded configuration values, sometimes including API keys
- Client-side access control logic revealing authorization models
- Library versions with known CVEs
- Source maps exposing original, unminified source code

None of this is visible to a source-code scanner unless it also models the build pipeline and deployment configuration. The shipped bundle is a different artifact from the source repository.

Black-box SPA analysis tools—which render applications in headless browsers, intercept network requests, and analyze JavaScript bundles—operate in this gap. They answer a different question: not “is this code vulnerable?” but “what has this application already exposed to every visitor?”

2.4 The Benchmark Gap

The security tooling landscape now contains at least four distinct categories:

1. **Traditional SAST/SCA** (Semgrep, Snyk, CodeQL)—pattern-matching on source code and dependency manifests
2. **Traditional DAST** (Burp Suite, ZAP)—probing running applications with known attack payloads
3. **AI-powered white-box** (Claude Code Security)—reasoning-based source code analysis
4. **AI-powered black-box** (SPA Hacking Agent)—autonomous client-side exposure analysis

No benchmark evaluates these categories against each other. No dataset captures vulnerability classes unique to agentic development. No scoring methodology accounts for the fundamental architectural differences between these approaches.

3 The AgentSecBench Framework

We propose a benchmark organized around four trust dimensions, inspired by the emerging trust infrastructure framework for agentic systems [3].

3.1 Design Principles

P1: Ground truth, not opinion. Every test case has a documented vulnerability with known type, location, severity, and correct fix. Scoring is automated.

P2: Modern stack, real patterns. Test applications use current frameworks (Next.js 15, Nuxt 4, SvelteKit) with realistic code patterns.

P3: Multi-perspective evaluation. The same vulnerability may be detectable from source or from the deployed bundle. The benchmark captures which perspective each tool operates from.

P4: Novel surfaces included. The benchmark includes agentic-specific vulnerability classes: MCP protocol security, skill registry poisoning, and AI-generated code artifacts.

P5: Open and extensible. Dataset, runners, and scoring

are open-source.

3.2 Dimension 1: Detection (SecureBench)

Core question: Does the tool find real vulnerabilities?

The test corpus consists of 60 intentionally vulnerable web applications in three tiers:

Table 1: Test corpus composition

Tier	Count	Examples
Standard	20	SQLi in Next.js API route, XSS in React component
Complex	20	IDOR via client-side route guard, race condition in payment
Agentic	20	Hallucinated package, malicious MCP tool, prompt injection in comments

Each application includes a ground truth manifest specifying vulnerability type, location (in both source and bundle), severity, detection perspective, and correct fix.

Metrics:

Table 2: Detection dimension metrics

Metric	Formula	Weight
True Positive Rate	found / total vulns	0.30
False Positive Rate	1 − (FP / total findings)	0.20
Severity Accuracy	correct sev / total found	0.15
Time to Detection	norm(1 / avg seconds)	0.15
Patch Quality	correct fixes / total fixes	0.20

3.3 Dimension 2: Quality (CompetenceBench)

Core question: Are the findings actionable and correct?

This dimension evaluates signal-to-noise ratio and practical utility through four tests:

1. **Precision under load:** Scan 10 large open-source projects (>100K LOC) with known CVE history. Measure findings against NVD ground truth.
2. **Adversarial robustness:** Inject prompt-injection attempts into code comments and variable names. Measure whether AI tools are manipulated into suppressing findings or generating false ones.
3. **Explanation quality:** Security engineers rate explanations on a 1–5 scale. Inter-rater reliability measured via Krippendorff’s α .
4. **Fix correctness:** Verify suggested patches compile, pass tests, and eliminate the vulnerability without introducing new ones.

3.4 Dimension 3: Auditability (Accountability-Bench)

Core question: Can we trace and reproduce what the tool did?

Table 3: Auditability criteria (scored 0–5 each)

Criterion	Description
Reasoning chain	Explains <i>why</i> , not just <i>that</i>
Evidence linking	Finding → code path → exploit scenario
Confidence calibration	Scores well-calibrated to actual accuracy
Output standardization	SARIF, CycloneDX, or structured schema
Reproducibility	Same input → same findings
Audit trail	Logs decision process for review

3.5 Dimension 4: Surface Coverage (SurfaceBench)

Core question: Does the tool cover the attack surfaces that matter in 2026?

This is the differentiation dimension. We test six surfaces:

Table 4: Attack surface coverage matrix

#	Surface	Trad.	AI-WB	AI-BB
S1	REST/GraphQL endpoints	~		
S2	Client-side secrets	×	~	
S3	Source map exposure	×	×	
S4	MCP protocol	×	~	×
S5	Skill/plugin registries	×	~	×
S6	Supply chain (agentic)	~		×

The columns represent expected capability based on architectural analysis. The benchmark replaces expectations with measurements.

3.6 Composite Scoring

The **AgentSec Score** is a weighted composite:

$$\text{Score} = 0.35D + 0.25Q + 0.15A + 0.25S \quad (1)$$

where D = Detection, Q = Quality, A = Auditability, S = Surface Coverage. Each is normalized to 0–100.

Tools are tagged with operating perspective: WB (white-box), BB (black-box), or HY (hybrid). A white-box tool is not penalized for missing source-map exposure, and a black-box tool is not penalized for missing business logic flaws only visible in source.

4 Benchmark Dataset Construction

4.1 Application Generation Methodology

Test applications are constructed using three methods:

Method 1: Manual seeding. Security engineers write realistic vulnerable applications. Each vulnerability is documented, peer-reviewed, and validated as exploitable.

Method 2: CVE reproduction. Known CVEs from real-world applications are reproduced in isolated test apps, ensuring the benchmark tests against actual production vulnerabilities.

Method 3: AI-assisted generation. AI agents generate applications with specific patterns. Engineers then seed vulnerabilities natural to the generated code structure, producing the “agentic” tier.

4.2 Framework Coverage

Table 5: Target SPA frameworks

Framework	Apps	Rationale
Next.js 15	20	Dominant, server components
Nuxt 4	10	Vue equivalent, different SSR
SvelteKit	10	Growing, different compilation
React SPA (Vite)	10	Pure client-side, max exposure
Angular 19	10	Enterprise, different modules

4.3 Vulnerability Taxonomy

We extend OWASP Top 10 [4] with agentic-specific categories:

Table 6: Extended vulnerability taxonomy

ID	Category	Source	Perspective
A01	Broken Access Control	OWASP	WB
A02	Cryptographic Failures	OWASP	WB
A03	Injection	OWASP	WB/BB
A05	Security Misconfiguration	OWASP	WB/BB
A06	Vulnerable Components	OWASP	WB/BB
AG01	Client-Side Secret Exposure	Ours	BB
AG02	Source Map Leakage	Ours	BB
AG03	MCP Tool Injection	Ours	WB
AG04	Skill Registry Poisoning	Ours	WB
AG05	Hallucinated Dependency	Ours	WB/SCA
AG06	Agent Prompt Manipulation	Ours	Both

5 Runner Architecture

5.1 Tool Integration

Each tool is wrapped in a standardized runner that: (1) accepts a test application (source directory or deployed URL), (2) invokes the tool with default configuration, (3) normalizes output to a common schema, and (4) records wall-clock time and resource consumption.

Listing 1: Runner directory structure

```
benchmark/
  apps/
    # Test applications
    nextjs-idor-001/
      src/
        # Source code
      deploy/
        # Build artifact
      ground-truth.yaml
  runners/
    base.py
    # Abstract interface
    claude_code_security.py # WB
    spa_hacking_agent.py   # BB
    semgrep.py             # WB
    snyk.py                 # WB
    burp_suite.py          # BB
    zap.py                  # BB
  scoring/
    detection.py | quality.py
    auditability.py | surface.py
    composite.py
```

5.2 Deployment Harness

For black-box tools, applications are deployed to isolated containers exposing accessible URLs. For white-box tools, the source directory is provided directly. This ensures each tool operates in its natural mode.

5.3 Time Budget

Each tool receives a fixed time budget: 2 minutes (small, <10 files), 5 minutes (medium, 10–50 files), or 10 minutes (large, 50+ files). Tools exceeding the budget receive partial credit for findings produced within the window.

6 Preliminary Analysis: Architectural Capability Mapping

Before running the benchmark, we predict structural strengths and weaknesses based on architecture.

6.1 White-Box AI (Claude Code Security)

Strengths: Reasons about code semantics; understands cross-function data flow; detects business logic flaws; suggests contextual fixes.

Blindspots: Cannot see build output (source-bundle gap); cannot detect runtime exposure; cannot assess the attacker’s external view; requires source access.

6.2 Black-Box AI (SPA Hacking Agent)

Strengths: Sees exactly what attackers see; detects client-side secret exposure; discovers source map leakage; enumerates deployed endpoints; identifies library versions with CVEs; no source access needed.

Blindspots: Cannot reason about server-side logic; limited to client-exposed surfaces; cannot suggest source-level fixes.

6.3 Traditional SAST and DAST

SAST strengths: Fast, deterministic, reproducible, extensive rule libraries. **SAST blindspots:** No semantic understanding, high FPR for complex vulns, blind to deployment.

DAST strengths: Tests running applications, detects runtime misconfigurations. **DAST blindspots:** Probe-based (only known patterns), limited JS analysis, slow.

6.4 The Complementarity Thesis

No single tool category provides complete coverage. We hypothesize:

1. AI white-box dominates Detection and Auditability
2. AI black-box dominates Surface (S1–S3)
3. Traditional tools show competitive speed but lower coverage
4. No tool scores above 70 across all four dimensions
5. White-box + black-box AI combination outscores any single tool

If confirmed, the agentic era requires **layered AI security**—not a single tool, but a composition of perspectives.

7 Implications for the Security Industry

7.1 The Trust Infrastructure Thesis

The benchmark dimensions map directly to emerging trust problems [3]:

Table 7: Trust layer alignment

Trust Problem	Benchmark Dimension
Secure: “Will this harm us?”	Detection
Competent: “Will it work?”	Quality
Accountable: “Can we prove it?”	Auditability
Verified: “Does it cover enough?”	Surface Coverage

7.2 Pricing Model Disruption

Traditional tools price per seat or per scan. Both break when agents are the primary developers. Per-seat pricing shrinks with headcount. Per-scan pricing explodes with continuous generation. The benchmark enables **per-vulnerability-class coverage** pricing: organizations select tools by coverage class, not scan volume.

7.3 The Composition Architecture

If the complementarity thesis holds, the winning architecture is a **composition layer**: routing source to white-box analysis, deployed apps to black-box reconnaissance, build artifacts to supply-chain analysis, and agent

configs to agentic surface analysis—then deduplicating, correlating, and prioritizing across all perspectives.

This “security orchestrator for agentic development” may be the most valuable product category the benchmark reveals.

8 Related Work

Agent Security Bench (ASB) [8] formalizes attacks and defenses on LLM-based agents with 400+ tools and 27 attack methods. ASB evaluates agent *safety*, not security *tool capability*—a complementary but distinct goal.

SEC-bench [9] benchmarks LLM agents on real vulnerability patching tasks, analogous to SWE-bench for security. It tests whether agents can *fix* vulnerabilities, not whether security tools can *find* them.

WASP [10] benchmarks web agent security against prompt injection attacks specifically. Its scope is narrower than AgentSecBench’s multi-surface, multi-tool evaluation.

SecureAgentBench [11] evaluates whether code agents generate secure code. It measures agent *output quality*, not detection tool capability.

AgentSecBench is the first benchmark designed for cross-category comparison of security *detection tools*—both AI-powered and traditional—across the full range of agentic-era attack surfaces.

9 Limitations and Future Work

Limitations: The initial 60-app dataset cannot cover all vulnerability classes. Auditability scoring relies partially on human evaluation. The benchmark evaluates tools in isolation, not in composition. MCP and skill registry surfaces (S4, S5) are based on threat modeling, not large-scale observed attacks.

Future work: (1) Quarterly dataset updates as agentic surfaces evolve. (2) An adversarial track evaluating evasion resistance. (3) Composition scoring for tool combinations. (4) A cost-effectiveness fifth dimension for ROI comparison.

10 Conclusion

The security industry is undergoing a structural transition. The audit layer that sustained two decades of tooling is being compressed out of existence by AI coding agents. New AI-powered tools—both white-box and black-box—offer capabilities traditional pattern-matching cannot match. But they operate from fundamentally different perspectives with complementary blindspots.

The industry needs a benchmark. Not a vendor benchmark designed to favor one product, but an open, community-maintained framework that quantifies what each tool catches, what it misses, and where the gaps remain.

AgentSecBench is that framework. By evaluating tools across detection capability, finding quality, auditability, and novel surface coverage, it provides the empirical foundation for security architecture decisions in the agentic era.

The audit layer is dying. The trust layer is emerging. The benchmark tells you who to trust—and what to verify.

References

- [1] Anthropic. “Making frontier cybersecurity capabilities available to defenders.” February 2026. <https://www.anthropic.com/news/claude-code-security>
- [2] C. E. Jimenez et al. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” *ICLR*, 2024.
- [3] P. Gosavi. “SaaS Cybersecurity is Dead.” 2026. As analyzed in “Follow the White Trust” (unpublished).
- [4] OWASP Foundation. “OWASP Top 10:2021.” <https://owasp.org/Top10/>
- [5] OASIS. “Static Analysis Results Interchange Format (SARIF) Version 2.1.0.” 2018.
- [6] CyberNews. “Amateur attacker uses AI toolkits, including Claude and Deepseek, to hack Fortinet firewalls.” February 2026.
- [7] NIST. “National Vulnerability Database.” <https://nvd.nist.gov/>
- [8] H. Zhang et al. “Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents.” *arXiv:2410.02644*, 2024.
- [9] “SEC-bench: Automated Benchmarking of LLM Agents on Real-World Software Security Tasks.” *arXiv:2506.11791*, 2025.
- [10] “WASP: Benchmarking Web Agent Security Against Prompt Injection Attacks.” *arXiv:2504.18575*, 2025.
- [11] “SecureAgentBench: Benchmarking Secure Code Generation under Realistic Vulnerability Scenarios.” *OpenReview*, 2025.